

Core Classes

This section describes the core classes of the Sports Manager application. These classes form the fundamental structure of the system and represent the main entities involved in the game. The design relies heavily on abstraction and object-oriented principles to ensure flexibility, modularity, and extensibility.

Each class has a clearly defined responsibility within the system, and together they create the core architecture that allows the application to simulate sports leagues and matches. By separating sport-specific logic from general game components, the system can support multiple sports without modifying the existing architecture.

ISport (Interface)

The ISport interface defines the general structure and rules that every sport must follow within the system. Since the application is designed to support multiple sports, it is important that the main game logic does not depend on any specific sport implementation.

The ISport interface provides a common contract that all sports must implement. This ensures that the rest of the application can interact with different sports in a consistent and predictable way.

Instead of writing sport-specific logic directly inside the game engine, the system communicates with the ISport interface. Concrete sport implementations provide the actual behavior.

The interface defines several important methods that describe the rules of a sport, including:

-getRequiredPlayers() – returns the number of players required in a team for the sport

-getMatchDuration() – returns the duration of a match

-getPointForWin() – returns the number of points awarded to a team for winning a match

By defining these rules in the interface, the system becomes highly flexible. Any new sport can be introduced simply by implementing the ISport interface.

This design follows the Open-Closed Principle, meaning the system is open for extension but closed for modification.

FootballSport

The FootballSport class is a concrete implementation of the ISport interface that defines the rules and characteristics of football.

This class provides football-specific information required by the game engine, such as the number of players per team, match duration, and scoring rules.

For example, football typically requires eleven players on the field and may follow a standard match duration format. These parameters are defined inside the FootballSport class.

Because the rest of the system interacts only with the ISport interface, the FootballSport implementation remains completely independent from the UI and game management components.

This separation ensures that football rules are encapsulated within their own class, making the code easier to maintain and extend.

HeadballSport

The HeadballSport class is another implementation of the ISport interface and represents the fictional sport used in the project example.

Although headball is not a real sport, the system treats it exactly the same way as football because both follow the same interface contract.

This class defines the specific rules of headball, including:

- number of players required for each team
- match duration
- scoring system

Because HeadballSport implements the same interface as FootballSport, the game engine can simulate headball matches without needing any special modifications.

This demonstrates the flexibility of the system architecture.

BasePlayer (Abstract Class)

The BasePlayer class is an abstract class that represents a generic player within the system. Since different sports may require different player characteristics, this class provides a shared structure that can be extended by sport-specific player types.

The BasePlayer class contains attributes that are common to all players, including:

- name** – the player's name
- age** – the player's age
- fitness** – a numerical value representing the player's physical condition
- isInjured** – a boolean value indicating whether the player is injured

In addition to these attributes, the class also defines general behaviors such as:

- train()** – improves the player's fitness level
- recover()** – helps the player recover from injuries or fatigue

Because BasePlayer is abstract, it cannot be instantiated directly. Instead, specific player types such as FootballPlayer and HeadballPlayer inherit from it.

This design promotes code reuse and ensures that common functionality is shared across all player types.

FootballPlayer

The FootballPlayer class extends the BasePlayer class and represents a player participating in football matches.

This class inherits all common player attributes from BasePlayer, including name, age, fitness, and injury status. Additional football-specific attributes could also be added if needed, such as:

- player position (goalkeeper, defender, midfielder, forward)

- skill ratings

- tactical roles

The FootballPlayer class allows the system to create football-specific players while still maintaining the shared structure defined in BasePlayer.

HeadballPlayer

The HeadballPlayer class also extends the BasePlayer class and represents players participating in headball matches.

Although the underlying attributes are inherited from BasePlayer, this class can introduce sport-specific properties if necessary.

This inheritance structure ensures that each sport can define its own player characteristics while still sharing common functionality.

BaseTeam

The BaseTeam class represents a team participating in the league. A team acts as a container that groups multiple players together and represents the competitive unit within matches.

Each team contains several attributes, including:

- teamName** – the name of the team

- players** – a list containing the players belonging to the team

- points** – the total points the team has earned in the league

Teams participate in matches and accumulate points based on match results. The BaseTeam class is also responsible for providing information about the team's roster and overall composition.

Because teams are central to match simulations and league standings, this class plays a crucial role in the system.

Match

The Match class represents a game played between two teams. It contains all the necessary information required to simulate and store the results of a match.

Key attributes of the Match class include:

- homeTeam** – the team playing at home
- awayTeam** – the opposing team
- homeScore** – the score achieved by the home team
- awayScore** – the score achieved by the away team
- isPlayed** – indicates whether the match has already been played

The class also contains the method:

playMatch()

This method is responsible for triggering the match simulation. Once the match is played, the scores are recorded and used to update league standings.

The Match class acts as the fundamental unit of competition within the league system.

MatchEngine

The MatchEngine class is responsible for simulating matches between teams. It acts as the core component that determines match outcomes.

The match simulation process takes into account several factors such as:

- the strength of the teams
- player fitness levels
- sport-specific rules
- random elements to simulate unpredictability

The MatchEngine interacts with the ISport interface to obtain sport-specific parameters. This ensures that the same match engine can simulate different sports without modification.

The main method provided by this class is:

simulateMatch(team1, team2, rules)

This method calculates the final match score based on the given teams and the rules defined by the sport.

By separating the simulation logic into a dedicated class, the system maintains a clean architecture and improves maintainability.

League

The League class represents the competition structure in which teams participate. It manages the list of teams, generates the match schedule, and keeps track of the league standings.

Important attributes include:

- teams** – the list of teams participating in the league
- fixtures** – the schedule of matches
- currentWeek** – indicates the current week of the season

The League class also provides methods such as:

- generateFixture()** – creates the match schedule
- updateStandings()** – updates team rankings after matches are played

During the season, teams play matches according to the fixture list. After each match, the league standings are updated based on the results.

The League class therefore serves as the central component responsible for managing the entire competition.